JavaScript

Concevoir des pages web interactives : bases, structures, DOM, événements, fonctions, dates et tableaux

Par **Harouna Kane**

Objectifs



Concevoir des pages web interactives



Connaître et pratiquer les nouveautés ES6



Interagir avec le DOM

Sommaire

Introduction

A. Présentation

Les bases

A. Intégration
B. Syntaxe générale
C. Les variables
D. Débugger

Manipuler les types simples

A. Opérateurs arithmétiques
B. Les nombres
C. Chaînes de caractères

Structures de base

A. Opérateurs de comparaison
B. Opérateurs logiques
C. Structures conditionnelles
D. Structures itératives

Les fonctions

A. Présentation et portée
B. Arguments et valeur de retour
C. Anonymes et auto-invoquées

Les tableaux

A. Accès et parcours
B. Ajouter, supprimer, modifier
C. Méthodes et tri

Sommaire

Les dates

A. Affichage et méthodes
B. Parser et comparer

L'objet window

A. Présentation et dimensions
B. Méthodes
C. Location

Le DOM

A. Présentation et objectifs
B. Les objets et leur contenu
C. Récupérer un objet
D. Attributs et style

Les événements

A. Présentation et intégration
B. Liste des événements
C. Écouteurs d'événements

INTRODUCTION

Présentation

> JavaScript est un langage de programmation créé en 1995. Il est exécuté directement par le navigateur, sans installation : c'est lui qui rend une page **interactive** (réagir à un clic, vérifier un formulaire, modifier la page sans la recharger). Depuis la norme **ES6** (2015), le langage s'est modernisé, et il fonctionne aussi côté serveur avec Node.js.

HTML

La structure : le contenu de la page.

CSS

La présentation : couleurs, mise en page.

JavaScript

Le comportement : actions et réactions.

LES BASES

Intégration

> Le code JavaScript s'ajoute à une page HTML avec la balise **<script>**. On peut l'écrire directement dans la page, mais la bonne pratique est de le placer dans un **fichier .js séparé**. L'attribut **defer** attend que le HTML soit chargé avant d'exécuter le script.

index.html

```
<!-- 1. Script interne -->
<script>
  alert("Bonjour !");
</script>

<!-- 2. Fichier externe (recommandé) -->
<script src="app.js" defer></script>
```

LES BASES

Syntaxe générale

> Un programme est une suite d'**instructions** terminées par un point-virgule. Les accolades **{ }** regroupent des instructions en **bloc**. JavaScript est **sensible à la casse** : nom ≠ Nom. Les commentaires ne sont pas exécutés : ils expliquent le code.

app.js

```
// Commentaire sur une ligne
/* Commentaire
   sur plusieurs lignes */

console.log("Instruction 1");
if (true) {             // début du bloc
  console.log("Dans le bloc");
}                       // fin du bloc
```

LES BASES

Les variables

> Une variable est une **boîte nommée** qui stocke une valeur pour la réutiliser. On la déclare avec **let** si sa valeur peut changer, avec **const** si elle ne changera pas (ES6). L'ancien mot-clé **var** est à éviter. Nommage : camelCase, sans accent ni espace, sans chiffre au début.

app.js

```
let age = 25;          // déclaration + affectation
age = 26;              // nouvelle valeur : OK

const TVA = 0.2;       // constante
TVA = 0.1;             // ❌ Erreur : const

let prenomUtilisateur = "Awa";
```

LES BASES

Les types de variables

> Chaque valeur a un **type**. JavaScript le devine tout seul (typage dynamique) : on ne l'écrit pas à la déclaration. L'opérateur **typeof** permet de connaître le type d'une variable.

Type	Exemple	typeof
Nombre	42, 3.14, -7	"number"
Chaîne	"Bonjour", 'Salut'	"string"
Booléen	true, false	"boolean"
Indéfini	let x;	"undefined"

app.js

```
let prix = 19.99;
console.log(typeof prix);    // "number"
```

LES BASES

Débugger

> Débugger, c'est trouver pourquoi le code ne fait pas ce qu'on attend. Les outils de développement du navigateur (**F12**) affichent les erreurs dans la **console**. On y écrit avec **console.log()** pour suivre les valeurs, et **debugger** met le programme en pause pour l'examiner pas à pas.

app.js

```
let total = 0;
console.log("total =", total);   // affiche une valeur
console.warn("Attention");      // avertissement jaune
console.error("Problème");      // erreur rouge

debugger;                 // pause dans les DevTools
```

MANIPULER LES TYPES SIMPLES

Opérateurs arithmétiques

> Ils effectuent des calculs : **+ − * /**, le **modulo %** (reste de la division) et la **puissance ****. **++** et **--** ajoutent ou retirent 1. Attention : avec une chaîne, **+** ne calcule pas, il **concatène** (colle les textes).

app.js

```
let a = 10, b = 3;
a + b;       // 13
a / b;       // 3.3333…
a % b;       // 1 (reste)
b ** 2;      // 9 (puissance)
a++;         // a vaut 11

"5" + 2;     // "52" (concaténation)
```

MANIPULER LES TYPES SIMPLES

Les nombres : base et extremums

> Un nombre peut s'écrire dans une autre **base** avec **toString(base)** : 2 pour le binaire, 16 pour l'hexadécimal. Les nombres ont des **limites** : au-delà de **Number.MAX_SAFE_INTEGER**, les calculs ne sont plus exacts.

app.js

```
// Changement de base
let n = 255;
n.toString(2);        // "11111111"
n.toString(16);       // "ff"
// Les extremums
Number.MAX_SAFE_INTEGER;  // 9007199254740991
Number.MIN_SAFE_INTEGER;  // -9007199254740991
1 / 0;                // Infinity
```

MANIPULER LES TYPES SIMPLES

Conversion et calculs

> Une saisie utilisateur est toujours du **texte**. Pour calculer, on la **convertit en nombre** avec Number(), parseInt() ou parseFloat(). L'objet **Math** fournit les fonctions de calcul : arrondis, minimum, maximum, nombre aléatoire…

app.js

```
Number("42");        // 42
parseInt("12px", 10);  // 12 (base 10)
parseFloat("3.5 kg");  // 3.5
Number("abc");       // NaN (pas un nombre)
Math.round(4.6);     // 5
Math.floor(4.9);     // 4
Math.max(3, 9, 1);    // 9
Math.random();       // entre 0 et 1
```

Chaînes : recherche

> Une chaîne est une suite de caractères, chacun repéré par sa **position** (index) qui commence à **0**. **length** donne le nombre de caractères. Pour chercher un mot : **indexOf()** renvoie sa position (ou −1 s'il est absent), **includes()** répond par true ou false.

app.js

```
const phrase = "Bonjour le monde";
phrase.length;            // 16
phrase[0];              // "B"
phrase.indexOf("le");      // 8
phrase.indexOf("chat");     // -1 (absent)
phrase.includes("monde");   // true
phrase.startsWith("Bon");   // true
```

Chaînes : extraction

> **slice(début, fin)** extrait une sous-chaîne (la position de fin est exclue). **trim()** enlève les espaces inutiles. Les **gabarits** ES6, entre accents graves (backtick en anglais), insèrent une variable avec **${ }**. Ces méthodes renvoient une nouvelle chaîne sans modifier l'originale.

app.js

```
const s = "Bonjour le monde";
s.slice(0, 7);        // "Bonjour"
s.slice(-5);          // "monde"
s.toUpperCase();      // "BONJOUR LE MONDE"
"  Awa  ".trim();     // "Awa"
const nom = "Awa";
`Bonjour ${nom} !`;   // "Bonjour Awa !"
```

Opérateurs de comparaison

> Une comparaison pose une question dont la réponse est un **booléen** : true ou false. **==** compare seulement la valeur (en convertissant les types), **===** compare la valeur **et** le type : c'est celui qu'on utilise.

Opérateur	Signification	Exemple
===	strictement égal	5 === 5 → true
!==	strictement différent	5 !== "5" → true
== / !=	égal / différent (valeur)	5 == "5" → true
< > <= >=	plus petit, plus grand…	7 >= 7 → true

app.js

```
const majeur = age >= 18;   // true ou false
```

STRUCTURES DE BASE

Opérateurs logiques

> Ils combinent plusieurs conditions. **&& (ET)** est vrai si **toutes** les conditions sont vraies. **|| (OU)** est vrai si **au moins une** est vraie. **! (NON)** inverse le résultat.

&&
true && false
→ false

||
true || false
→ true

!
!true
→ false

app.js

```
const age = 20, permis = true;
age >= 18 && permis;     // true : peut conduire
age < 12 || age > 65;  // false : pas de réduction
!permis;               // false
```

STRUCTURES DE BASE

Conditions : if, else if, else

> Une condition permet d'exécuter un bloc **seulement si** un test est vrai. **if** teste une première condition, **else if** en teste une autre si la précédente est fausse, et **else** s'exécute quand aucune n'est vraie. Un seul bloc est exécuté.

app.js

```
const note = 12;
if (note >= 10) {
  console.log("Admis");
} else if (note >= 8) {
  console.log("Rattrapage");
} else {
  console.log("Ajourné");
}
```

STRUCTURES DE BASE

Conditions : switch

> **switch** compare une valeur à une liste de cas possibles. Plus lisible qu'une suite de else if quand on teste **une seule variable**. **break** sort du switch ; **default** s'exécute si aucun cas ne correspond.

app.js

```
switch (jour) {
  case "samedi":
  case "dimanche":
    message = "Week-end";
    break;
  default:
    message = "Semaine";
}
```

STRUCTURES DE BASE

Boucles : while et do-while

> Une boucle **répète** un bloc tant qu'une condition est vraie. **while** teste la condition **avant** chaque tour : le bloc peut ne jamais s'exécuter. **do-while** la teste **après** : le bloc s'exécute au moins une fois. La condition doit finir par devenir fausse, sinon la boucle est infinie.

while

```
let i = 1;
while (i <= 3) {
  console.log(i);
  i++;
}
// 1, 2, 3
```

do-while

```
let rep;
do {
  rep = prompt("Oui ?");
} while (rep !== "oui");
// redemande
// tant que ≠ "oui"
```

STRUCTURES DE BASE

Boucles : for et for in

> **for** s'utilise quand on connaît le nombre de tours. Ses trois parties : **initialisation** ; **condition** ; **incrément**. **for in** parcourt les **propriétés d'un objet** : à chaque tour, la variable reçoit le nom d'une propriété.

for

```
for (let i = 0; i < 3; i++) {
  console.log(i);
}
// 0, 1, 2
```

for in

```
const u = { nom: "Awa" };
for (const cle in u) {
  console.log(cle, u[cle]);
}
// nom Awa
```

Présentation

> Une **fonction** est un bloc de code **nommé et réutilisable**. On la **déclare** une fois avec le mot-clé function, puis on l'**appelle** autant de fois que nécessaire en écrivant son nom suivi de parenthèses. Elle évite de répéter le même code.

app.js

```
// 1. Déclaration
function afficherBienvenue() {
  console.log("Bienvenue sur le site");
}
// 2. Appels
afficherBienvenue();
afficherBienvenue();    // réutilisable à volonté
```

LES FONCTIONS UTILISATEURS

Portée des variables

> La **portée** indique où une variable est utilisable. Déclarée hors de toute fonction, elle est **globale** : visible partout. Déclarée dans une fonction ou un bloc avec let ou const, elle est **locale** : elle n'existe qu'à l'intérieur.

app.js

```
const site = "Simplon";       // globale
function saluer() {
  const message = "Bonjour";  // locale
  console.log(message, site);  // ✔ les deux
}

console.log(message);   // ✖ ReferenceError
```

Les arguments

> Les **paramètres** sont des variables déclarées entre les parenthèses de la fonction. À l'appel, on leur donne des valeurs : les **arguments**. Il peut y en avoir **un** ou **plusieurs**. ES6 permet une **valeur par défaut**.

app.js

```
// Un argument, avec valeur par défaut
function saluer(prenom = "inconnu") {
  console.log(`Bonjour ${prenom}`);
}
saluer("Awa");   // Bonjour Awa
saluer();        // Bonjour inconnu
function presenter(nom, age) { console.log(nom, age); }
presenter("Moussa", 32);
```

Valeur de retour

> Avec **return**, une fonction **renvoie un résultat** à l'endroit où elle a été appelée : on peut le stocker dans une variable ou l'utiliser dans un calcul. return **arrête** aussi la fonction : le code placé après n'est pas exécuté.

app.js

```
function calculerTTC(prixHT) {
  return prixHT * 1.2;
}

const prix = calculerTTC(100);     // 120
const total = calculerTTC(50) + 5;  // 65
console.log(calculerTTC(10));    // 12
```

Fonctions anonymes et fléchées

> Une fonction **anonyme** n'a pas de nom : on la range dans une variable ou on la passe directement en argument (tableaux, minuteries, événements…). La **fonction fléchée** (ES6) est une écriture plus courte avec **=>** ; sur une seule ligne, le return est implicite.

app.js

```
// Anonyme rangée dans une variable
const carre = function (n) {
  return n * n;
};
// Même chose en fléchée (ES6)
const carre2 = (n) => n * n;
carre2(4);                    // 16
```

Fonctions auto-invoquées

> Une fonction **auto-invoquée** (IIFE) s'exécute **immédiatement**, dès qu'elle est définie, sans être appelée ailleurs. On l'entoure de parenthèses, puis on ajoute **()** à la fin. Elle sert à isoler des variables pour ne pas encombrer la portée globale.

app.js

```
(function () {
  const secret = "invisible dehors";
  console.log("Exécutée tout de suite");
})();

// Version fléchée
(() => console.log("Init"))();
```

LES TABLEAUX

Présentation et accès

> Un **tableau** stocke une **liste ordonnée** de valeurs dans une seule variable, entre crochets **[ ]**. Chaque élément a un **index** qui commence à **0**. **length** donne le nombre d'éléments ; le dernier s'obtient avec at(-1) (ES2022).

app.js

```
const fruits = ["mangue", "banane", "orange"];

fruits[0];              // "mangue"
fruits[2];              // "orange"
fruits.length;           // 3
fruits.at(-1);           // dernier : "orange"
fruits[5];              // undefined
```

Les tableaux sont des objets

> En JavaScript, un tableau est un **objet** particulier : typeof renvoie "object", on le teste avec **Array.isArray()**. Un objet classique range ses valeurs par **clé** (nom) au lieu d'un index ; on y accède avec le **point** ou les **crochets**.

app.js

```
typeof [1, 2];           // "object"
Array.isArray([1, 2]);   // true

const eleve = { nom: "Diallo", note: 15 };
eleve.nom;               // "Diallo"
eleve["note"];           // 15
eleve.ville = "Creil";   // ajoute une propriété
```

LES TABLEAUX

Parcourir un tableau

> **Parcourir**, c'est traiter chaque élément l'un après l'autre. Trois façons : **for** avec l'index, **for...of** (ES6) qui donne chaque valeur, et **forEach** qui appelle une fonction pour chaque élément.

app.js

```
const notes = [12, 15, 9];
for (let i = 0; i < notes.length; i++) {
  console.log(i, notes[i]);
}
for (const note of notes) {
  console.log(note);
}
notes.forEach((note) => console.log(note));
```

LES TABLEAUX

Ajouter, supprimer, modifier

> **push** ajoute à la fin, **unshift** au début. **pop** retire le dernier élément, **shift** le premier. **splice(index, nombre)** supprime à n'importe quelle position. Pour **modifier**, on affecte une nouvelle valeur à un index.

app.js

```
const t = ["b", "c"];
t.push("d");        // ["b", "c", "d"]
t.unshift("a");     // ["a", "b", "c", "d"]
t.pop();           // ["a", "b", "c"]
t.shift();         // ["b", "c"]
t[0] = "B";         // ["B", "c"]
t.splice(1, 1);     // ["B"]
```

LES TABLEAUX

Méthodes de tableaux

> Les méthodes modernes évitent d'écrire des boucles. **map** transforme chaque élément, **filter** garde ceux qui passent un test, **find** renvoie le premier trouvé. **includes** et **indexOf** recherchent, **join** assemble en texte. map et filter renvoient un **nouveau tableau**.

app.js

```
const prix = [10, 25, 40];
prix.map((p) => p * 2);      // [20, 50, 80]
prix.filter((p) => p > 20);  // [25, 40]
prix.find((p) => p > 20);    // 25
prix.includes(40);           // true
prix.indexOf(25);            // 1
prix.join(" - ");            // "10 - 25 - 40"
```

Tri des tableaux

> **sort()** trie le tableau, par défaut **comme du texte** : pour des chaînes, c'est l'ordre alphabétique. Pour des **nombres**, il faut lui donner une **fonction de comparaison**, sinon 100 passe avant 25. **reverse()** inverse l'ordre.

app.js

```
const noms = ["Moussa", "Awa", "Fatou"];
noms.sort();              // ["Awa", "Fatou", "Moussa"]

const n = [100, 25, 3];
n.sort();                 // [100, 25, 3] ❌
n.sort((a, b) => a - b);    // [3, 25, 100] croissant
const desc = n.toSorted((a, b) => b - a);  // copie (ES2023)
```

LES DATES

Présentation et affichage

> L'objet **Date** représente un instant précis. **new Date()** sans argument donne la date et l'heure actuelles. Pour l'affichage, **toLocaleDateString()** et **toLocaleTimeString()** formatent selon la langue choisie. Attention : les **mois sont numérotés de 0 à 11**.

app.js

```
const maintenant = new Date();
const noel = new Date(2026, 11, 25);  // 11 = décembre

noel.toLocaleDateString("fr-FR");   // "25/12/2026"
noel.toLocaleDateString("fr-FR", {
  weekday: "long", day: "numeric", month: "long"
});                    // "vendredi 25 décembre"
```

LES DATES

Méthodes de dates

> Les méthodes **get…** lisent une partie de la date (année, mois, jour, heure…) et les méthodes **set…** la modifient. Pratique pour construire un format personnalisé ou pour ajouter des jours à une date.

app.js

```
const d = new Date(2026, 9, 20);     // 20 octobre 2026
d.getFullYear();            // 2026
d.getMonth();               // 9 (octobre)
d.getDate();                // 20 (jour du mois)
d.getDay();                 // 2 (mardi ; 0 = dimanche)
d.setDate(d.getDate() + 7);   // + 7 jours
`${d.getDate()}/${d.getMonth() + 1}`;  // "27/10"
```

LES DATES

Parser et comparer les dates

> **Parser**, c'est transformer un texte en date : on passe la chaîne au format **"AAAA-MM-JJ"** à new Date(). Pour **comparer**, on utilise < et > ; pour une différence, **getTime()** donne la date en millisecondes, qu'on convertit ensuite en jours.

app.js

```
const debut = new Date("2026-10-20");
const fin = new Date("2026-10-22");

debut < fin;                    // true
const ms = fin.getTime() - debut.getTime();
const jours = ms / (1000 * 60 * 60 * 24);  // 2
```

L'OBJET WINDOW

Présentation et dimensions

> **window** représente l'onglet du navigateur. C'est l'**objet global** : tout ce qui n'est pas rangé ailleurs lui appartient, d'où le fait qu'on peut écrire alert() au lieu de window.alert(). Il donne notamment les **dimensions** de la zone d'affichage, en pixels.

app.js

```
window.innerWidth;     // largeur visible, ex. 1280
window.innerHeight;    // hauteur visible
screen.width;          // largeur de l'écran

if (window.innerWidth < 768) {
  console.log("Affichage mobile");
}
```

L'OBJET WINDOW

Méthodes : dialogues et timing

> Les **boîtes de dialogue** affichent un message (alert), demandent une saisie (prompt) ou une confirmation (confirm). Les **méthodes de timing** lancent du code plus tard : **setTimeout** une seule fois, **setInterval** en boucle. Durées en millisecondes.

app.js

```
alert("Bienvenue !");
const nom = prompt("Votre nom ?");   // texte ou null
const ok = confirm("Continuer ?");  // true / false

setTimeout(() => alert("2 s écoulées"), 2000);
const id = setInterval(() => console.log("tic"), 1000);
clearInterval(id);           // arrête la boucle
```

L'OBJET WINDOW

Location et history

> **location** contient l'**adresse (URL)** de la page en cours : on peut la lire, la découper, rediriger vers une autre page ou recharger. **history** permet de naviguer dans l'historique, comme les boutons Précédent et Suivant.

app.js

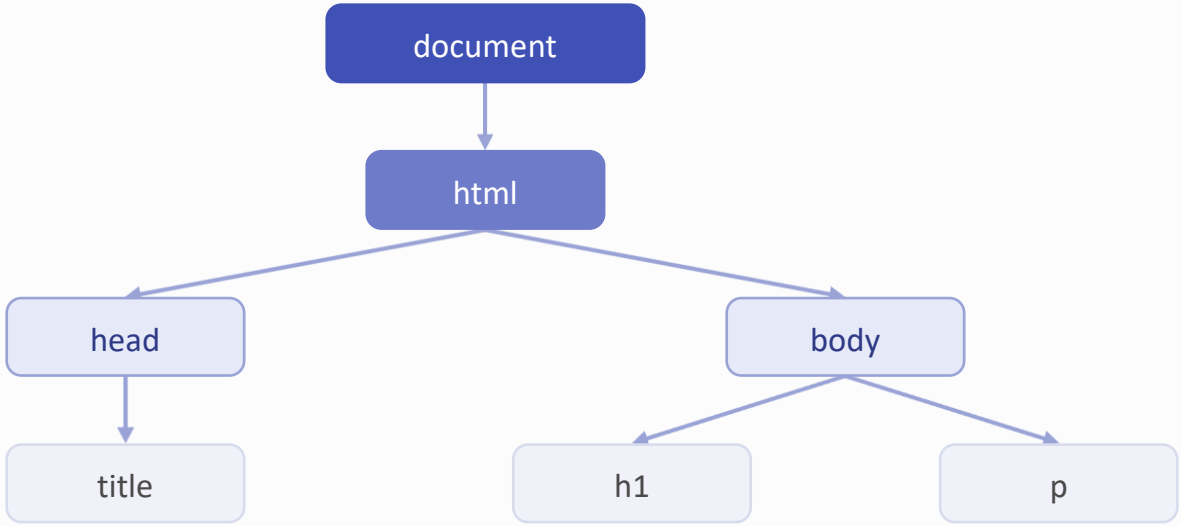
```
location.href;      // "https://site.fr/page?id=3"
location.hostname;  // "site.fr"
location.search;    // "?id=3"
location.href = "contact.html";  // redirection
location.reload();    // recharger

history.back();       // page précédente
history.forward();    // page suivante
```

LE DOM

Présentation et objectifs

> Le **DOM** (Document Object Model) est la représentation de la page HTML sous forme d'**arbre d'objets**. Chaque balise devient un objet (un « nœud ») que JavaScript peut **lire, modifier, ajouter ou supprimer**. Le point d'entrée est l'objet **document**.



LE DOM

Les objets et leur contenu

> Une fois un élément récupéré, on peut lire ou changer son **contenu**. **textContent** manipule le texte brut ; **innerHTML** interprète les balises HTML. On peut aussi **créer** un nouvel élément et l'ajouter dans la page.

app.js

```
const titre = document.querySelector("h1");
titre.textContent;              // lire le texte
titre.textContent = "Nouveau titre";
titre.innerHTML = "Bonjour <em>Awa</em>";

const p = document.createElement("p");
p.textContent = "Paragraphe ajouté";
document.body.append(p);
```

LE DOM

Récupérer un objet

> Avant d'agir sur un élément, il faut le **récupérer** dans une variable. On peut le cibler par son **id**, sa **balise**, sa **classe**, ou par n'importe quel **sélecteur CSS** avec querySelector (le premier trouvé) et querySelectorAll (tous).

app.js

```
document.getElementById("menu");       // par id
document.getElementsByTagName("p");  // par balise
document.getElementsByClassName("carte");

document.querySelector("#menu");       // le 1er
document.querySelector("input[name=email]");
document.querySelectorAll("ul > li");    // tous
```

Les attributs d’un objet

> Les attributs HTML (src, href, alt, value…) se lisent avec **getAttribute()** et se modifient avec **setAttribute()**. Les plus courants sont aussi accessibles directement comme des **propriétés** de l’objet. Pour un champ de formulaire, **value** donne la saisie.

app.js

```
const img = document.querySelector("img");
img.getAttribute("src");          // "chat.jpg"
img.setAttribute("src", "chien.jpg");
img.alt = "Un chien";           // via la propriété

const champ = document.querySelector("#email");
champ.value;                  // texte saisi
```

Le style d'un objet

> La propriété **style** modifie le CSS d'un élément, en camelCase (background-color → backgroundColor). Bonne pratique : préparer des classes dans le CSS et les gérer avec **classList** (add, remove, toggle, contains).

app.js

```
const box = document.querySelector(".box");
box.style.color = "white";
box.style.backgroundColor = "#3F51B5";

box.classList.add("actif");
box.classList.remove("cache");
box.classList.toggle("ouvert");   // ajoute ou retire
box.classList.contains("actif"); // true
```

Présentation et intégration

> Un **événement** est une action qui se produit dans la page : un clic, une touche, un chargement, une saisie… JavaScript peut **réagir** à ces événements en exécutant du code. Première façon de faire : l'attribut HTML **onclick**, simple mais qui mélange HTML et JavaScript.

index.html

```
<!-- Dans le HTML (à éviter) -->
<button onclick="alert('Clic !')">OK</button>
<!-- Avec une propriété en JS -->
<script>
  const btn = document.querySelector("button");
  btn.onclick = () => alert("Clic !");
</script>
```

Liste des événements courants

> Chaque événement a un **nom**, qu'on utilise pour y réagir. Voici les plus utilisés, classés par famille.

Famille	Événement	Se déclenche quand…
Souris	click, dblclick	on clique, double-clique
Souris	mouseover, mouseout	le pointeur entre, sort
Clavier	keydown, keyup	une touche est enfoncée, relâchée
Formulaire	input, change	la valeur d'un champ change
Formulaire	submit	le formulaire est envoyé
Formulaire	focus, blur	un champ gagne, perd le focus
Page	load, DOMContentLoaded	la page est chargée
Fenêtre	resize, scroll	on redimensionne, on défile

LES ÉVÉNEMENTS

Écouteur d'événement

> La méthode recommandée est **addEventListener(nom, fonction)** : elle « écoute » un élément et lance la fonction à chaque fois que l'événement se produit. La fonction reçoit en **argument** un objet **event** qui décrit ce qui s'est passé (élément cliqué, touche…).

app.js

```
const btn = document.querySelector("#valider");
btn.addEventListener("click", (event) => {
  console.log("Bouton cliqué");
  console.log(event.target);   // l'élément cliqué
});

document.addEventListener("keydown", (e) =>
  console.log(e.key));
```

Imbrication et suppression

> Quand des éléments sont **imbriqués**, un clic sur l'enfant remonte aussi vers ses parents : c'est la **propagation**. **stopPropagation()** l'arrête. Pour **supprimer** un écouteur avec removeEventListener, la fonction doit avoir un **nom**.

app.js

```
enfant.addEventListener("click", (e) => {
  e.stopPropagation();      // le parent ne réagit pas
});
function direBonjour() {
  alert("Bonjour");
}
btn.addEventListener("click", direBonjour);
btn.removeEventListener("click", direBonjour);
```

MISE EN PRATIQUE

À vous de jouer : liste de tâches

1. Créer une page avec un champ texte, un bouton « Ajouter » et une liste ul vide.
2. Au clic, récupérer la saisie (value) et l'ajouter dans un tableau de tâches.
3. Écrire une fonction afficher() qui reconstruit la liste à partir du tableau.
4. Refuser une tâche vide avec une condition et un message d'alerte.
5. Afficher la date du jour en haut de la page, au format français.
6. Bonus : supprimer une tâche au clic et trier la liste par ordre alphabétique.